

Hi3519DV500 eMMC 分区表配置指南

1. 概述

Hi3519DV500 平台使用 eMMC 存储，系统通过 `emmc_burn_table.xml` 定义分区布局，通过 `fs.yml` 定义各文件系统镜像大小。两者必须对齐，否则烧写失败或运行异常。

涉及文件

文件	作用	位置
<code>emmc_burn_table.xml</code>	定义 eMMC 分区地址和大小	<code>device/soc/hisilicon/hi3519dv500/burn/emmc_burn_table.xml</code>
<code>fs.yml</code>	定义 rootfs/userdata 镜像内容及大小	<code>vendor/hisilicon/hispark_hi3519dv500_linux/fs.yml</code>
<code>rootfsimg_linux.sh</code>	实际生成 ext4 镜像的脚本	<code>build/lite/make_rootfs/rootfsimg_linux.sh</code>

产物流向



2. emmc_burn_table.xml 字段详解

2.1 XML 结构

```
<Partition_Info>
  <Part Sel="1" PartitionName="boot"      FlashType="emmc" FileSystem="none" Start="0x0"
  Length="0x80000"   SelectFile="boot_image.bin" />
  <Part Sel="1" PartitionName="env"        FlashType="emmc" FileSystem="none" Start="0x80000"
  Length="0x80000"   SelectFile="emmc_env.bin" />
  <Part Sel="1" PartitionName="bl31"        FlashType="emmc" FileSystem="none" Start="0x100000"
  Length="0x80000"   SelectFile="bl31.bin" />
  <Part Sel="1" PartitionName="kernel"      FlashType="emmc" FileSystem="none" Start="0x180000"
  Length="0xd00000"  SelectFile="uImage-fdt" />
  <Part Sel="1" PartitionName="rootfs"      FlashType="emmc" FileSystem="ext4" Start="0xe80000"
  Length="0xc800000" SelectFile="rootfs_ext4.img" />
  <Part Sel="1" PartitionName="userdata"    FlashType="emmc" FileSystem="ext4"
  Start="0xd680000" Length="0xa00000"   SelectFile="userdata_ext4.img" />
</Partition_Info>
```

2.2 字段说明

字段	类型	说明
<code>Se1</code>	枚举	1=烧写此分区，0=跳过此分区
<code>PartitionName</code>	字符串	分区名称，烧写工具按名称匹配
<code>FlashType</code>	字符串	存储介质类型，本平台固定 <code>emmc</code>
<code>FileSystem</code>	字符串	文件系统： <code>ext4</code> =ext4文件系统， <code>none</code> =裸二进制数据
<code>Start</code>	十六进制	分区在 eMMC 上的 起始字节地址
<code>Length</code>	十六进制	分区的 字节大小
<code>SelectFile</code>	字符串	要烧写到该分区的镜像文件名

3. 地址计算方法

3.1 基本单位

所有地址和长度单位均为**字节（Byte）**，使用十六进制表示。

1 KB = 0x400	(1,024 字节)
1 MB = 0x100000	(1,048,576 字节)

3.2 常用大小速查表

大小	十六进制	十进制
512 KB	0x80000	524,288
1 MB	0x100000	1,048,576
2 MB	0x200000	2,097,152
4 MB	0x400000	4,194,304
8 MB	0x800000	8,388,608
10 MB	0xA00000	10,485,760
13 MB	0xD00000	13,631,488
16 MB	0x1000000	16,777,216
32 MB	0x2000000	33,554,432
64 MB	0x4000000	67,108,864
96 MB	0x6000000	100,663,296

大小	十六进制	十进制
100 MB	0x6400000	104,857,600
128 MB	0x8000000	134,217,728
200 MB	0xC800000	209,715,200
256 MB	0x10000000	268,435,456
300 MB	0x12C00000	314,572,800
512 MB	0x20000000	536,870,912

3.3 换算公式

十六进制 -> MB： $0xC800000 / 0x100000 = 0xC8 = 200 \text{ MB}$

MB -> 十六进制： $200 * 0x100000 = 0xC800000$

3.4 核心规则：地址必须严格连续

分区[0] Start = 0x0（固定）

分区[N] Start = 分区[N-1] Start + 分区[N-1] Length

每个分区的 Start 必须等于前一个分区的 End（Start + Length），不能重叠，不能留空洞。

4. 当前分区表计算过程

4.1 逐分区推导

+-----+
| eMMC 地址空间（字节） |
| 0x0 -> 0xE080000 (224MB) |
+-----+
| boot | env | b131 | kernel | rootfs | userdata |
| 512KB | 512KB | 512KB | 13MB | 200MB | 10MB |
+-----+
0x0	0x80000	0x100000	0x180000	0xe80000	0xd680000
+	+	+	+	+	+
0x80000	0x80000	0x80000	0xd00000	0xc800000	0xa00000
=	=	=	=	=	=
0x80000	0x100000	0x180000	0xe80000	0xd680000	0xe080000
+-----+

#	分区	Start	计算过程	Length	大小	End
1	boot	0x0	固定起始	0x80000	512KB	0x80000
2	env	0x80000	0x0 + 0x80000	0x80000	512KB	0x100000

#	分区	Start	计算过程	Length	大小	End
3	bl31	0x100000	0x80000 + 0x80000	0x80000	512KB	0x180000
4	kernel	0x180000	0x100000 + 0x80000	0xd00000	13MB	0xe80000
5	rootfs	0xe80000	0x180000 + 0xd00000	0xc800000	200MB	0xd680000
6	userdata	0xd680000	0xe80000 + 0xc800000	0xa00000	10MB	0xe080000

总占用：0xE080000 = 224 MB

4.2 验证命令

```
# 验证地址连续性（在容器内执行）
printf "rootfs End = 0x%X\n" $((0xe80000 + 0xc800000))
# 输出：rootfs End = 0xD680000

printf "userdata End = 0x%X\n" $((0xd680000 + 0xa00000))
# 输出：userdata End = 0xE080000
```

5. fs.yml 与分区表的对齐关系

5.1 fs.yml 中的镜像大小定义

```
# rootfs: 第三个参数 200 表示 200MB
fs_make_cmd:
  - ${root_path}/build/lite/make_rootfs/rootfsimg_linux.sh ${fs_dir} ext4 200

# userdata: 第三个参数 10 表示 10MB
fs_make_cmd:
  - ${root_path}/build/lite/make_rootfs/rootfsimg_linux.sh ${fs_dir} ext4 10
```

5.2 对齐规则

fs.yml 镜像大小 <= emmc_burn_table.xml 分区 Length
必须满足

镜像	fs.yml 参数	镜像实际大小	XML 分区 Length	是否匹配
rootfs	ext4 200	200 MB	0xc800000 = 200 MB	匹配
userdata	ext4 10	10 MB	0xa00000 = 10 MB	匹配

5.3 rootfsimg_linux.sh 镜像大小计算逻辑

```
EMMC_ROOTFS_SIZE=$3 # 从 fs.yml 传入，如 200
COUNT_SIZE=$((EMMC_ROOTFS_SIZE * 1024 * 2)) # 200 * 1024 * 2 = 409600
dd if=/dev/zero of=${ROOTFS_IMG} bs=512 count=${COUNT_SIZE}
# 最终镜像 = 512 * 409600 = 209,715,200 = 200 MB
```

6. 调整分区大小的完整步骤

场景示例：将 rootfs 从 200MB 调整为 300MB

步骤1：确定目标大小

```
目标 rootfs = 300 MB
300 * 0x100000 = 0x12C00000
```

步骤2：计算新的分区地址

只修改 rootfs 的 Length 和其后所有分区的 Start (boot/env/bl31/kernel 不动)：

```
rootfs:
  Start = 不变 = 0xe80000
  Length = 0x12C00000 (300MB)
  End    = 0xe80000 + 0x12C00000 = 0x1AE80000

userdata:
  Start = 0x1AE80000 (= rootfs End)
  Length = 不变 = 0xa00000 (10MB)
  End    = 0x1AE80000 + 0xa00000 = 0x1B880000
```

验证命令：

```
printf "rootfs End = 0x%X\n" $((0xe80000 + 0x12C00000))
# 输出: rootfs End = 0x1AE80000

printf "userdata End = 0x%X\n" $((0x1AE80000 + 0xa00000))
# 输出: userdata End = 0x1B880000
```

步骤3：检查 eMMC 总容量

```
总占用 = 0x1B880000 = 462 MB
需要 eMMC >= 512 MB (常见规格)
```

步骤4: 修改 fs.yml

```
# vendor/hisilicon/hispark_hi3519dv500_linux/fs.yml
# 将 rootfs 的第三个参数从 200 改为 300
fs_make_cmd:
  - ${root_path}/build/lite/make_rootfs/rootfsimg_linux.sh ${fs_dir} ext4 300
```

步骤5: 修改 emmc_burn_table.xml

```
<!-- device/soc/hisilicon/hi3519dv500/burn/emmc_burn_table.xml -->

<!-- rootfs: Length 从 0xc800000 改为 0x12c00000 -->
<Part Sel="1" PartitionName="rootfs" FlashType="emmc" FileSystem="ext4"
    Start="0xe80000" Length="0x12c00000" SelectFile="rootfs_ext4.img" />

<!-- userdata: Start 从 0xd680000 改为 0x1ae80000 -->
<Part Sel="1" PartitionName="userdata" FlashType="emmc" FileSystem="ext4"
    Start="0x1ae80000" Length="0xa00000" SelectFile="userdata_ext4.img" />
```

步骤6: 全量编译验证

```
# 全量编译验证
rm -rf out && hb build -f
```

7. 快速参考

调整口诀

改分区，记三步：

1. 改 Length -> 定新大小

2. 算新 Start -> 本分区End = Start + Length, 下个分区Start = 本分区End

3. 推后面 -> 修改点之后所有分区 Start 都要重新算

双文件同步：

- fs.yml -> 控制镜像文件大小（第三个参数，单位 MB）

- XML -> 控制分区大小（Length，单位字节十六进制）

换算公式：

- N MB = N x 0x100000

- 0xABCDEF / 0x100000 = ? MB

常见错误

错误提示	原因	解决
分区的地址必须大于已使用分区的大小	分区 Start < 前一个分区的 End, 地址重叠	重新计算 Start, 确保 Start = 前一分区 Start + Length

错误提示	原因	解决
烧写失败/镜像过大	fs.yml 镜像大小 > XML 分区 Length	同步增大 XML 中的 Length
空洞/未使用空间	分区之间 Start 不连续	确保每个 Start = 前一个 End

8. 历史修改记录

日期	修改前	修改后	说明
2026-05-16	rootfs Length=0x6000000 (96MB), userdata Start=0x6e80000	rootfs Length=0xC800000 (200MB), userdata Start=0xd680000	rootfs 分区与 fs.yml 中 200MB 镜像大小对齐